

Тема 4. Работа с файловой системой (синхронная)



Хекслет Колледж

Цель занятия:

Научить студентов читать и записывать файлы в Node.js с помощью синхронных методов модуля fs.

Учебные вопросы:

1. Модуль fs - общее знакомство
2. Чтение файла - fs.readFileSync()
3. Запись файла - fs.writeFileSync()
4. Дополнительные синхронные методы fs
5. Практика

1. Модуль fs - общее знакомство

fs (File System) - встроенный модуль Node.js для работы с файловой системой. С его помощью можно читать, записывать, удалять и переименовывать файлы и папки прямо из программы.

Подключение:

```
const fs = require('fs');
```

Синхронный vs асинхронный подход

Модуль fs предоставляет два варианта работы с файлами - синхронный и асинхронный.

Синхронный - программа читает файл и ждёт пока операция завершится, только потом идёт дальше:

```
console.log('Начало');  
const data = fs.readFileSync('file.txt', 'utf8'); // ждём  
console.log(data);  
console.log('Конец');
```

Вывод:

Начало

содержимое файла

Конец

Асинхронный - программа запускает чтение файла и не ждёт
- сразу идёт дальше, а когда файл прочитан - возвращается к результату:

```
console.log('Начало');  
fs.readFile('file.txt', 'utf8', (err, data) => {  
  |   console.log(data); // выполнится позже  
});  
console.log('Конец');
```

Вывод:

Начало

Конец

содержимое файла ← пришло позже

Асинхронный подход подробно разберём в следующей лекции.

Сравнение подходов:

	Синхронный	Асинхронный
Ожидание	Программа ждёт	Программа не ждёт
Простота кода	Проще	Сложнее
Производительность	Ниже	Выше
Название методов	<code>readFileSync</code>	<code>readFile</code>
Когда использовать	Простые скрипты, запуск программы	Серверы, много операций

Синхронные методы подходят когда:

- Программа запускается один раз и завершается - например скрипт обработки файлов
- Нужно прочитать конфигурацию при старте программы
- Пишем простую утилиту командной строки
- Учимся - синхронный код проще читать и понимать

2. Чтение файла - `fs.readFileSync()`

`fs.readFileSync` - это синхронный метод Node.js для чтения данных из файла.

Синтаксис:

```
fs.readFileSync(path, encoding)
```

`path` - Путь к файлу

`encoding` - Кодировка - обычно 'utf8'

Пример:

Создадим файл hello.txt с содержимым:

Привет, Node.js!

Читаем его:

```
const fs = require('fs');  
const path = require('path');  
  
const filePath = path.join(__dirname, 'hello.txt');  
const data = fs.readFileSync(filePath, 'utf8');  
  
console.log(data); // Привет, Node.js!
```

Что такое кодировка utf8?

При чтении файла нужно указать кодировку - это способ интерпретации байтов файла как текста.

```
const data = fs.readFileSync('hello.txt', 'utf8');
console.log(data);           // Привет, Node.js!
console.log(typeof data);    // string

// без кодировки – получаем Buffer (сырые байты)
const buffer = fs.readFileSync('hello.txt');
console.log(buffer);         // <Buffer d0 9f d1 80 d0 b8 ...>
console.log(typeof buffer);  // object
```

Обработка ошибок - try/catch

Если файл не существует - программа
выбросит ошибку и завершится:

```
const data = fs.readFileSync('несуществующий.txt', 'utf8');  
// Error: ENOENT: no such file or directory
```

Чтобы программа не падала - оборачиваем в **try/catch**:

```
const filePath = path.join(__dirname, 'hello.txt');

try {
  const data = fs.readFileSync(filePath, 'utf8');
  console.log(data);
} catch (err) {
  console.log('Файл не найден:', err.message);
}
```

Теперь если файл не существует - программа выведет сообщение об ошибке и продолжит работу.

Чтение нескольких файлов

```
const files = ['first.txt', 'second.txt', 'third.txt'];

files.forEach(file => {
  try {
    const filePath = path.join(__dirname, 'data', file);
    const data = fs.readFileSync(filePath, 'utf8');
    console.log(`=== ${file} ===`);
    console.log(data);
  } catch (err) {
    console.log(`Файл ${file} не найден`);
  }
});
```

ИТОГ:

Синтаксис

```
fs.readFileSync(path, 'utf8')
```

Возвращает

Строку с содержимым файла

Без кодировки

Возвращает Buffer (сырые байты)

Файл не найден

Выбрасывает ошибку `ENOENT`

Обработка ошибок

```
try/catch
```

3. Запись файла - `fs.writeFileSync()`

`fs.writeFileSync` - это синхронный метод Node.js для записи данных в файл.

Синтаксис:

```
fs.writeFileSync(path, data, options);
```

`path` - путь к файлу (строка).

`data` - данные для записи (строка или Buffer).

`options` - (необязательно) объект с кодировкой, режимом доступа или флагами.

Базовое использование (Перезапись)

По умолчанию метод работает в режиме «перезаписи». Если файл уже существует, его содержимое будет удалено и заменено новым. Если файла нет, Node.js создаст его автоматически.

```
const content = 'Привет, студенты! Это запись в файл.';
const filePath = path.join(__dirname, 'note.txt');

try {
  fs.writeFileSync(filePath, content, 'utf8');
  console.log('Файл успешно сохранен по адресу:', filePath);
} catch (err) {
  console.error('Ошибка при записи:', err);
}
```

Использование флагов (Flags)

Флаги определяют поведение системы при открытии файла.

Они передаются в третьем аргументе в виде объекта.

Основные флаги:

- 'w' (Write): Значение по умолчанию. Открывает для записи (перезапись).
- 'a' (Append): Открывает для добавления. Данные пишутся в конец файла.

Пример использования флага 'a' - дозапись:

```
const logPath = path.join(__dirname, 'logs.txt');
const logMessage = `Новая запись в лог: ${new Date().toLocaleTimeString()}\n`;

try {
  // Используем флаг 'a' для дозаписи (Append)
  fs.writeFileSync(logPath, logMessage, { flag: 'a' });
  console.log('Данные успешно добавлены в лог');
} catch (err) {
  console.error('Ошибка при ведении лога:', err);
}
```

Флаг	Поведение если файл есть	Поведение если файла нет
w' (по умолчанию)	Перезаписывает (удаляет старое)	Создает новый файл
'a'	Добавляет в конец	Создает новый файл

Важные нюансы для разработчика:

- Создание директорий: Метод `writeFileSync` может создать файл, но не умеет создавать папки. Если путь указывает в несуществующую папку, программа выдаст ошибку `ENOENT`.
- Типы данных: Если передать объект или массив напрямую, вы получите строку `[object Object]`. Всегда используйте `JSON.stringify(data)` перед записью сложных структур.
- Блокировка потока: Пока идет запись (особенно больших файлов), весь Node.js процесс "замирает". Это допустимо в скриптах инициализации, но критично для высоконагруженных серверов.

4. Дополнительные синхронные методы fs

Помимо чтения и записи, модуль fs предоставляет инструменты для манипуляции файлами и директориями. Все перечисленные ниже методы являются синхронными (блокирующими).

Проверка существования: `fs.existsSync()`

Единственный метод в этом списке, который не выбрасывает ошибку, если объект не найден, а просто возвращает `true` или `false`.

```
const targetPath = path.join(__dirname, 'config.json');

if (fs.existsSync(targetPath)) {
  console.log('Файл найден, можно работать!');
} else {
  console.log('Файл отсутствует. ');
}
```

Создание папки: `fs.mkdirSync()`

Создает новую директорию.

Нюанс: Если папка уже существует, метод выбросит ошибку.

```
const dirPath = path.join(__dirname, 'my_folder');

try {
  if (!fs.existsSync(dirPath)) {
    fs.mkdirSync(dirPath);
    console.log('Папка создана');
  }
} catch (err) {
  console.error('Ошибка при создании папки:', err);
}
```


Список содержимого: `fs.readdirSync()`

Возвращает массив строк - имен всех файлов и папок внутри указанной директории.

```
const folderPath = __dirname;

try {
  const files = fs.readdirSync(folderPath);
  console.log('Содержимое текущей папки:', files);
} catch (err) {
  console.error('Не удалось прочитать папку:', err);
}
```

Переименование и перемещение: **fs.renameSync()**
Универсальный метод. Если пути до и после одинаковые - это переименование. Если разные - перемещение.

```
const oldPath = path.join(__dirname, 'old_name.txt');
const newPath = path.join(__dirname, 'new_name.txt');

try {
  fs.renameSync(oldPath, newPath);
  console.log('Файл успешно переименован/перемещен');
} catch (err) {
  console.error('Ошибка перемещения:', err);
}
```

Удаление файла: `fs.unlinkSync()`

Удаляет файл по указанному пути. Не работает с папками (для папок используется `fs.rmSync` или `fs.rmdirSync`).

```
const fileToDelete = path.join(__dirname, 'temp.log');

try {
  if (fs.existsSync(fileToDelete)) {
    fs.unlinkSync(fileToDelete);
    console.log('Файл удален');
  }
} catch (err) {
  console.error('Ошибка удаления:', err);
}
```

ИТОГ:

Метод	Задача	Что возвращает
<code>existsSync(path)</code>	Проверить наличие	<code>true / false</code>
<code>mkdirSync(path)</code>	Создать директорию	<code>undefined</code>
<code>readdirSync(path)</code>	Список файлов/папок	<code>Array</code> (массив строк)
<code>renameSync(old, new)</code>	Переименовать/Переместит	<code>undefined</code>
<code>unlinkSync(path)</code>	Удалить файл	<code>undefined</code>

5. Практика - работаем с файлами

Кейс: Чтение и изменение содержимого.

Типичная задача в разработке - считать файл, изменить данные в памяти и сохранить результат обратно.

Сценарий: У нас есть файл counter.txt, где хранится число. Нам нужно увеличить его на 1.

```
const filePath = path.join(__dirname, 'counter.txt');

try {
  // 1. Читаем файл (как строку 'utf8')
  let data = fs.readFileSync(filePath, 'utf8');

  // 2. Преобразуем строку в число и инкрементируем
  let count = parseInt(data) || 0;
  count++;

  // 3. Записываем обратно
  fs.writeFileSync(filePath, count.toString(), 'utf8');

  console.log(`Счетчик обновлен: ${count}`);
} catch (err) {
  console.error('Произошла ошибка при обновлении счетчика:', err.message);
}
```

Типичные ошибки (Troubleshooting)

Ошибка **ENOENT** (Error NO ENTry)

Что значит: Файл или папка по указанному пути не найдены.

Причина: Ошибка в названии файла или попытка записи в папку, которой не существует.

Решение: Проверить путь через **console.log(filePath)** и наличие родительских папок.

Ошибка **EACCES / EPERM** (Permission Denied)

Что значит: Отказано в доступе.

Причина: Попытка записи в системную папку (например, в корень диска C: или /root) или файл открыт другой программой в эксклюзивном режиме.

Решение: Запустить редактор/терминал от имени администратора или проверить права доступа к папке.

Ошибка **EISDIR** (Is a Directory)

Что значит: Ожидался файл, но обнаружена директория.

Причина: Вы вызвали **fs.readFileSync** на путь, который является папкой.

Решение: Уточнить путь, добавив имя конкретного файла.

Проблема `[object Object]` в файле

Что значит: Логическая ошибка разработчика.

Причина: Попытка записать массив или объект напрямую в `writeFileSync` без конвертации.

Решение: Всегда использовать `JSON.stringify(data)` перед записью структур данных.

Золотые правила "Выживания":

- Всегда `try...catch`: Синхронные методы «убивают» весь процесс при любой ошибке. Без блока `try...catch` ваше приложение упадет даже из-за опечатки в имени файла.
- Проверка перед действием: Перед тем как читать файл или удалять его, используйте `fs.existsSync()`. Это предотвратит 90% фатальных ошибок.
- Логируйте пути: Если что-то не работает, первым делом выведите в консоль полный путь, созданный через `path.join(__dirname, ...)`. В 8 из 10 случаев проблема в лишнем или недостающем слеше.

Выводы:

- Модуль fs - это «ворота» к файловой системе сервера. Без него Node.js ограничен только оперативной памятью.
- Синхронность = Блокировка. Методы с суффиксом *Sync останавливают выполнение всего скрипта.
- Абсолютные пути - это стандарт. Использование `path.join(__dirname, 'file.txt')` гарантирует, что ваш код запустится на любой ОС и из любой рабочей директории.
- Кодировка имеет значение.
- Безопасность через `try...catch..`
- Флаги - гибкость записи. По умолчанию файлы перезаписываются ('w'), но флаг 'a' (append) превращает запись в дополнение, что критично для логов.

Контрольные вопросы

1. В чем главная опасность использования `fs.readFileSync` в коде веб-сервера, который обрабатывает сотни запросов в секунду?
2. Зачем нам нужен модуль `path` и метод `path.join()`, если можно просто написать `'./data/file.txt'`?
3. Что вернет `fs.readFileSync(path)`, если вторым аргументом не передать кодировку `'utf8'`?
4. Как дописать строку в конец существующего файла, не удаляя его старое содержимое?
5. Какую ошибку выдаст Node.js, если вы попытаетесь прочитать файл, которого не существует, и как предотвратить падение программы в этом случае?
6. Можно ли с помощью `fs.writeFileSync` создать файл в папке, которая еще не создана?